Professional CV

1. Personal Information

Sridhar Samineni

Software Engineer · Systems Architecture · Security · Cloud · Applied AI

25+ years of hands-on engineering across systems software, enterprise platforms, distributed infrastructure, and emerging AI technologies.

Location: Herndon, VA
Email: SSamineni009@gmail.com
Phone: 703.459.0473
Profiles: GitHub · AgentNebula

---

2. Professional Profile

Software engineer and technology-focused product developer with **over 25 years of hands-on experience** designing, implementing, integrating, and validating software across embedded systems, network security, enterprise platforms, distributed data systems, cloud infrastructure, and applied AI. Experience ranges from **Linux kernel drivers, IPsec/IKE, cryptographic protocols, and PKI** to **scalable server architectures, Hadoop/Cassandra analytics, machine-learning pipelines, and AI-agent infrastructure**.

Strong record of translating technical requirements into working systems through **independent research, architecture design, open-source evaluation, rapid prototyping, and implementation**. Contributions include router security and hardware-assisted IPsec processing; smart-card middleware and authentication; mobile VPN gateways; certificate-authority integration; distributed server and data platforms; performance-testing frameworks; and operational analytics. Work spans low-level C/C++ development, Java enterprise systems, and Python-based AI/ML applications.

More recently, expanded into **domain-specific natural-language-to-SQL model training, XGBoost-based anomaly detection, and AI-agent security and interoperability**. Developed AgentNebula as an independent engineering project to explore agent registration, discovery, lifecycle management, identity, OAuth, PKI, and policy-based authorization using A2A, AuthZEN, and OPA.

Engineering strengths: Ability to move between unfamiliar technology domains, identify and reuse mature open-source components, define clear integration boundaries, and build and troubleshoot complex software from protocol and infrastructure layers through application interfaces. Emphasis on maintainable architecture, security, performance, and practical delivery rather than technology adoption for its own sake.

3. Technical Expertise

Engineering Breadth: Over 25 years of hands-on software engineering across embedded systems, Linux kernel and networking, cryptography and PKI, enterprise applications, distributed data platforms, cloud infrastructure, and applied machine learning. Experience spans requirements analysis, technology evaluation, architecture, prototyping, implementation, integration, performance validation, deployment, and operational troubleshooting. Particularly experienced in selecting and integrating established open-source components to build maintainable systems without unnecessary custom infrastructure.

> **Software Architecture & Product Engineering** — End-to-end design and implementation of security-sensitive and data-intensive applications; modular and layered architectures; REST APIs and service integration; extensible frameworks and plugin interfaces; high availability, scalability, performance testing, and technical prototyping. Experience developing complete software components from initial investigation through integration, packaging, and validation.

Programming & Development — Python, C, C++, Java, SQL/PLSQL, shell scripting (Bash, awk, sed), Perl, JavaScript, PHP; multithreading, sockets, JNI, Linux system APIs, Java enterprise applications, test automation, and API development.

Linux, Embedded Systems & Networking — Linux kernel modules and device drivers, Netfilter/iptables hooks, network protocol implementation, embedded Linux and customized bootable images, Linux administration, process and application tuning, TCP/IP diagnostics, packet analysis, VPN clients and gateways, MobileIP, IPsec/IKE, L2TP, and network appliances. Tools include tcpdump, Wireshark, netstat/ss, and nmap.

Security, Identity & Cryptography — PKI and X.509 certificate lifecycle, certificate authorities (EJBCA, Entrust, OpenCA evaluation), PKCS#8/#10/#11/#12, OpenSSL, TLS/mTLS, OAuth 2.0, JWT-based security, authentication and authorization, smart cards and ISO 7816 APDUs, Linux PAM, Kerberos, Active Directory, LDAP, RADIUS, SCEP/OCSP investigation, IPsec/IKEv1/IKEv2, hardware-assisted cryptography, and role-based access control.

AI Agents & Agent Infrastructure — Independent AgentNebula platform engineering: agent registration, discovery and lifecycle management; official A2A Python SDK integration; agent identity and security integration; OAuth services; policy evaluation with AuthZEN, Open Policy Agent (OPA), and Rego; PKI and trust infrastructure; agent orchestration and skill-selection experiments. MCP integration is an area of ongoing investigation, not a claimed completed implementation.

Machine Learning & Natural-Language Systems — Domain-specific natural-language-to-SQL model training and inference; synthetic training-data generation, schema-aware prompts, linguistic augmentation, FLAN-T5 fine-tuning with Hugging Face Transformers and PEFT/LoRA; PyTorch; XGBoost regression-based anomaly detection, Isolation Forest, feature engineering, threshold calibration, model comparison, and MLflow experiment tracking.

Cloud, Containers & Deployment Engineering — AWS-related application and ML deployment engineering, including SageMaker, Lambda, API Gateway, S3, and Terraform infrastructure definitions; Docker-based packaging; Kubernetes and OpenStack experience. Cloud deployment experience should be differentiated by project during the final review; repository infrastructure definitions are not presented as evidence of production operation.

Distributed Data, Big Data & Databases — Hadoop ecosystem (HDFS, MapReduce, Hive and UDFs, Sqoop, Flume, Pig, ZooKeeper, Oozie), Hadoop cluster setup and administration, Oracle-to-Cassandra data migration, Cassandra and HBase application development, PostgreSQL, MySQL, Oracle, SQLite, memcachedb, data ingestion pipelines, operational analytics, and reporting.

Enterprise Applications & Integration — Java/J2EE, Spring and Spring Expression Language, GWT and MVP/mvp4g, REST/JSON, JSP, GlassFish, Tomcat, Apache HTTP Server, database integration, certificate-management interfaces, enterprise reporting, and service-oriented application design. Experience with HornetQ and Hazelcast.

Performance, Reliability & Engineering Tools — Load and stress testing with JMeter and Grinder; Java profiling with JProfiler; Linux/JVM/application-server tuning; Linux Virtual Server and load-balancer evaluation; network and system troubleshooting; Rundeck workflow automation; Ant/RPM build and packaging; Git-based development, test environments, and reproducible deployment workflows.

Engineering Approach: Research-driven, implementation-focused engineering combining low-level technical depth with system-wide architecture. Comfortable evaluating unfamiliar protocols, frameworks, and open-source packages, building proof-of-concept implementations, identifying integration boundaries, and developing working solutions across multiple technology stacks. This approach has been applied repeatedly—from cryptographic drivers and router security to scalable server platforms, distributed analytics, and AI-agent infrastructure.

4. Applied AI, Machine Learning, and Independent Engineering Projects

This section presents independent, hands-on engineering work undertaken to extend established experience in enterprise software, distributed systems, networking, and security into **AI-agent infrastructure, machine learning, and natural-language applications**. Projects cover research and standards evaluation, architecture, implementation, integration, testing, packaging, and deployment. The focus is on building reusable software and evaluating emerging technologies against established protocols and open-source implementations.

The descriptions distinguish implemented capabilities from experimental integrations and ongoing investigation. For the EPP SLA projects, repository implementations and deployment configurations are documented without implying production adoption or independently measured business outcomes.

4.1 AgentNebula — Enterprise AI Agent Hub and Security Platform

Project Website: https://agentnebula.ai
Technical Documentation: https://docs.agentnebula.ai

Project Overview

Independently architected and developed **AgentNebula**, a modular enterprise AI-agent hub providing shared infrastructure for registering, discovering, securing, governing, and managing independently developed AI agents. The project addresses the challenge of integrating agents built with different frameworks and operated by different providers: each agent retains its own business logic while the platform supplies common registry, identity, trust, authentication, authorization, lifecycle, and administrative services.

AgentNebula is designed around **standards-based interoperability, service separation, and reuse of mature open-source components**. Its developer-facing packages support A2A agents without requiring application developers to recreate protocol handling and security mechanisms. The project includes hosted platform services, client integration libraries, example agents, user interfaces, documentation, and deployment tooling. It is an independent engineering project; enterprise customer adoption or quantified production outcomes are not claimed.

Architecture and Major Components

Core / Agent Registry — Agent registration, Agent Card metadata, publisher/provider relationships, catalog and capability information, and platform administration APIs.

Agent Discovery — Agent lookup and capability-oriented discovery through registry information and published agent metadata.

Resource Lifecycle — Resource identity, ownership relationships, lifecycle states, and administrative operations for managed agent resources.

Identity and Authentication — Account and client identities, authentication integration, credentials, and protected platform operations.

OAuth Server — OAuth authorization-server capabilities, client management, token issuance, and integration with centralized authorization decisions.

Policy Server — Policy decision service using **Open Policy Agent (OPA)** and **Rego**, with an **AuthZEN**-based authorization interface.

PKI and Trust — Certificate-authority functions, X.509 agent certificates, trust bundles, and TLS/mTLS-related integration.

Gateway and APIs — REST-based platform interfaces, gateway routing, and service integration.

Audit, Health, and Telemetry — Audit events, health monitoring, diagnostic information, and operational observability capabilities.

AgentNebula SDK — Reusable Python security primitives and integration components for OAuth, JOSE, token validation, and sender-constrained authentication.

AgentNebula Platform — Pluggable middleware connecting developer-facing agent applications, the official A2A Python SDK, and security/integration components.

AgentNebula SecureAgent — Configuration-driven developer interface for building A2A agents while separating business skills from protocol and security infrastructure.

SecureAgent Examples — Executable A2A/A2UI agent examples and integration scenarios, including standalone and platform-integrated demonstrations.

A2UI Rich Catalog — Reusable JavaScript/TypeScript visual components for agent-driven A2UI presentation, extending the basic catalog where richer semantics are needed.

Portal, Explorer, and Playground — Administration and onboarding, agent discovery and interaction, and agent experimentation/development interfaces.

Build, Installer, and Documentation — Container packaging, configuration, repeatable deployment workflows, and architecture/developer documentation.

Engineering Contributions

- **End-to-end platform architecture:** Designed service boundaries for agent registry and discovery, lifecycle management, identity, authentication, OAuth, authorization policy, PKI, administration, and observability; integrated the services through documented APIs and configuration.
- **Agent interoperability:** Adopted the **official A2A Python SDK** for A2A protocol behavior, rather than maintaining a parallel custom protocol implementation. Worked with Agent Cards, skills/capability metadata, agent invocation, and agent-facing integration boundaries.
- **OAuth and token security:** Built OAuth-related services and reusable client/resource-server security components, including token handling, validation, and integration with protected resources and policy decisions. Investigated and integrated modern OAuth security mechanisms such as sender-constrained access using **DPoP**.
- **Centralized authorization:** Implemented a policy service backed by **OPA/Rego** and exposed through **AuthZEN** conventions, separating authorization decisions from the services that enforce them. Explored agent-specific policy requirements for A2A and MCP scenarios; emerging agent-specific policy standards remain an investigation area.
- **Identity and PKI:** Developed agent and organizational identity management, certificate issuance and trust-management capabilities, and TLS/mTLS integration points.
- **Reusable developer components:** Developed and published **AgentNebula SDK**, **AgentNebula Platform**, and **SecureAgent** as layered Python packages: security primitives, pluggable agent middleware, and a simplified developer-facing interface.
- **Developer examples and testing:** Published executable examples and integration scenarios for A2A and A2UI workflows, with examples that can be run independently of the hosted registry as well as platform-connected scenarios.
- **Dynamic agent UI:** Investigated A2UI protocol integration and developed the **AgentNebula A2UI Rich Catalog**, a reusable npm package for richer agent-driven UI components.
- **Operational tooling:** Built platform administration and discovery interfaces, diagnostic/trace capabilities, service packaging, deployment configuration, and supporting technical documentation.
- **Open-source evaluation:** Evaluated and integrated existing standards implementations and open-source packages to reduce custom code, preserve clear boundaries, and avoid duplicating mature protocol functionality.

Protocols, Standards, and Technical Skills

The inventory below records technologies applied in AgentNebula or investigated as part of its design. Items explicitly marked investigated are not claims of completed production integration.

AI agents and interoperability — Agent2Agent (**A2A**) protocol, official A2A Python SDK, Agent Cards, agent capabilities and skills, A2A client/server integration, agent registration and discovery, agent lifecycle management; **MCP** policy/integration requirements (investigated).

Agent presentation — Agent-to-User Interface (**A2UI**), Basic and Rich Catalog concepts, structured UI payloads, React-oriented component rendering, JavaScript/TypeScript, npm package publication.

OAuth and API security — **OAuth 2.0**, OAuth authorization-server/client/resource-server responsibilities, bearer tokens, JWT, JOSE, JWK/JWKS, protected-resource metadata, client authentication, token validation, **DPoP (RFC 9449)**, **Resource Indicators (RFC 8707)**, **JWT Access Token Profile (RFC 9068)**, **private_key_jwt**.

Authorization and policy — **AuthZEN**, **Open Policy Agent (OPA)**, **Rego**, policy-as-code, policy decision and enforcement separation, centralized authorization, contextual policy evaluation, agent-oriented policy design.

Identity, trust, and cryptography — **PKI**, **X.509**, certificate authorities, certificate signing and issuance, certificate lifecycle, trust bundles, **TLS/mTLS**, digital signatures, credential management.

Architecture and APIs — Python, modular/service-oriented architecture, REST/JSON APIs, API gateways, pluggable middleware, configuration-driven components, asynchronous agent integration, resource ownership and lifecycle models.

Data and infrastructure — PostgreSQL, relational schema design, Docker, Linux, Nginx, Cloudflare Tunnel, OCI VM deployment, configuration templates, deployment automation.

Engineering and distribution — Git/GitHub, Python packaging, **PyPI**, **uv**, **npm**, automated/integration testing, MkDocs, API and architecture documentation, open-source dependency evaluation.

Published Developer Components

agent-nebula-sdk — PyPI · GitHub | Standards-oriented Python security SDK.

agent-nebula-platform — PyPI · GitHub | Pluggable A2A/security middleware.

agent-nebula-secureagent — PyPI · GitHub | Simplified interface for secure A2A agent development.

agent-nebula-secureagent-examples — PyPI · GitHub | Executable agent examples and integration tests.

@agent-nebula-npm/a2ui-rich — npm | Reusable A2UI Rich Catalog components.

The hosted Core, OAuth, Policy, and other internal platform services are described as platform components; their source repositories are not presented as publicly available.

4.2 EPP SLA — Natural Language to SQL Model Training

Description: Developed a domain-specific Text-to-SQL training pipeline that translates natural-language questions about Extensible Provisioning Protocol (EPP) service-level metrics into SQL queries. The project uses schema-aware prompts and synthetic training examples to investigate specialized model adaptation for operational analytics.

Technical Contributions:

- Implemented a configurable **synthetic question/SQL-pair generator** using EPP database schemas, query templates, filtering and aggregation rules, and SQLite-based SQL validation.
- Developed **linguistic data augmentation** and schema serialization to produce varied training questions while retaining database identifiers and SQL semantics.
- Implemented fine-tuning of **Google FLAN-T5** using **Hugging Face Transformers** and **PEFT/LoRA**, with a training/evaluation split and versioned model artifacts.
- Built schema-aware inference logic for generating SQL from user questions, including tokenization-aware handling of qualified column names.
- Created an end-to-end workflow for data generation, augmentation, model training, inference testing, and optional **Amazon S3** artifact synchronization; included **AWS SageMaker** deployment infrastructure in the repository.

Skills / Technologies: Python, PyTorch, FLAN-T5, Hugging Face Transformers, PEFT/LoRA, natural-language processing, Text-to-SQL, SQLite, Pandas, Amazon S3, AWS SageMaker, Terraform.

4.3 EPP SLA — AI Business Intelligence Agent

Description: Developed an interactive analytics application that accepts natural-language questions about EPP transaction volumes, response times, failures, registrar activity, and release impacts; generates SQL through a selected language-model backend; and presents query results as tables and visualizations.

Technical Contributions:

- Integrated a custom Text-to-SQL model alongside configurable **OpenAI** and **Hugging Face** model endpoints, enabling comparison of generated SQL and inference latency.
- Developed a **Streamlit** interface for question entry, SQL generation, query execution, tabular results, and interactive analytics.
- Implemented a **SQLite**-based EPP analytics dataset and synthetic-data generation for demonstrating operational reporting scenarios.
- Added configurable model endpoint authentication and optional **Amazon S3** database synchronization.

Skills / Technologies: Python, Streamlit, SQLite, Pandas, Plotly, REST APIs, Text-to-SQL, language-model integration, Amazon S3.

4.4 EPP SLA — Machine Learning Anomaly Detection

Description: Designed a multi-model anomaly-detection system for hourly EPP service-level metrics, including successful and failed transaction volumes and response-time measurements. The system evaluates deviations from expected operating patterns and supports comparison of supervised and unsupervised approaches.

Technical Contributions:

- Developed an **XGBoost regression-based anomaly detection approach** that estimates expected metric values and identifies deviations through residual analysis and configurable thresholds.
- Implemented shared feature engineering using **cyclical hour-of-day encoding** and **one-hot encoding of EPP commands**, with feature alignment between training and inference.
- Added an **Isolation Forest** approach for unsupervised anomaly scoring and cross-model comparison.
- Implemented **percentile-based threshold calibration**, alert-rate tuning, model-version configuration, and model-specific inference pipelines.
- Integrated **MLflow** experiment tracking and **FastAPI** inference endpoints; added AWS deployment configuration using **SageMaker, Lambda, API Gateway, and Terraform**.
- Developed a companion **Streamlit dashboard** for single-model analysis, side-by-side model comparison, MLflow experiment review, and simulated real-time alert visualization.

Skills / Technologies: Python, XGBoost, scikit-learn, Pandas, NumPy, MLflow, FastAPI, Streamlit, AWS SageMaker, AWS Lambda, API Gateway, Terraform, Docker.

5. Professional Experience

5.1 Verisign, Inc.

Principal Responsibilities and Engineering Scope

- **Registry SLA analysis and root-cause investigation:** Analyzed production EPP transaction logs, command response-time distributions, release histograms, registrar-specific traffic patterns, transaction volumes, and service-level deviations. Correlated **gateway SLA, application SLA, database SLA and system metrics, and network latency** to isolate likely sources of degradation and identify abnormal registrar activity.
- **End-to-end technical product ownership:** Gathered requirements; designed architecture and database models; implemented features and integrations; maintained and enhanced applications; automated deployments and reporting workflows; troubleshot operational issues; and produced technical and operational documentation.
- **Production-informed performance testing:** Converted observed registry traffic characteristics into representative workload scenarios for supported TLDs, including **.com, .net, .cc, and .tv**, and analyzed throughput limits, latency deviations, breaking points, and release regressions or improvements.
- **Reporting and analytics engineering:** Developed automated pipelines, extensible Hive reports, relational data models, and Grafana dashboards providing multidimensional views by registrar, TLD, IP version, gateway type, EPP command, and software release.
- **Infrastructure and database modernization:** Supported the evolution from virtual-machine deployments to **OpenStack** with **Terraform/Ansible**, then to **Verisign Kubernetes Services (VKS)** with manifests and Helm charts; migrated an approximately **800 GB** reporting database from **MySQL to PostgreSQL**, followed by deployment on **CloudNativePG**.

Projects and Technical Contributions

5.1.1 Shared SLA Data Processing and Reporting Architecture

Description: Architected and implemented a reusable, automated reporting framework used by **Sirius** for production SLA analysis and **PSTools** for performance-testing analytics. Both products share the underlying processing, report distribution, validation, persistence, and visualization patterns, with product-specific sources, reporting logic, and PostgreSQL schemas.

Reporting Pipeline: Sirius processes daily production SLA logs in Hadoop using automated, pluggable Hive queries and UDFs. PSTools uses the same reporting framework to process hourly performance-test SLA logs and metrics in the test Hadoop environment, with additional test and release-analysis reports. Generated reports are published to OpenStack Swift, where automated monitors detect, download, validate, and load them into PostgreSQL for Grafana visualization. A separate Sirius workflow ingests hourly database metrics from Swift. Both products use standardized PostgreSQL table conventions and product-specific schemas.

Technical Contributions:

- Designed the complete reporting workflow, from log availability and scheduled data processing through report publication, automated ingestion, persistence, and dashboard visualization.
- Created a **pluggable Hive reporting architecture**: adding a new report required adding a directory with its Hive query, without modifying the core orchestration workflow.
- Developed Hive queries and user-defined functions for multiple SLA report categories, supporting investigation of traffic anomalies, service-level deviations, and performance trends.
- Automated Hive report generation when the corresponding source SLA logs became available in Hadoop.
- Implemented report-monitoring services to detect newly published files in **OpenStack Swift**, download and validate them, and load results into PostgreSQL.
- Implemented a separate **hourly database-metrics ingestion path**, with Swift monitoring, download, validation, and database loading.
- Designed the **common PostgreSQL reporting data model**, standardizing shared fields and table conventions across reporting categories; represented each reporting product through its own PostgreSQL schema while retaining the reusable structure.
- Extended the shared framework with performance-testing-specific reports and release-comparison analytics without duplicating the underlying ingestion and storage architecture.

Skills / Technologies: Hadoop, HDFS, Hive, Hive UDFs, SQL, OpenStack Swift, PostgreSQL, data modeling, ETL, automated ingestion, extensible reporting frameworks, Grafana, distributed data processing.

5.1.2 Sirius — Production Registry SLA Analytics and Reporting

Description: Production registry SLA reporting platform maintaining approximately **15 years of historical SLA metrics**. Sirius supports multidimensional production analysis across registrars, IP versions, gateway types, TLDs, EPP commands, and software releases, with correlation of application, gateway, database, and network performance indicators.

Technical Contributions:

- Developed and maintained the production reporting pipeline: **daily SLA logs → Hadoop → automated Hive reports → Swift → report monitors → PostgreSQL → Grafana**.
- Implemented numerous pluggable Hive reports for SLA measurements and category-specific analysis, including views useful for identifying registrar anomalies and investigating performance deviations.
- Automated the processing of available production logs and end-to-end report generation and ingestion.
- Implemented **hourly database metrics** processing: metrics published to Swift were detected, downloaded, validated, and loaded into PostgreSQL through a dedicated monitor.
- Developed historical reporting and Grafana dashboards to compare trends, distributions, EPP command response-time histograms, and release behavior.
- Investigated potential sources of SLA degradation by correlating registrar traffic with **gateway, application, database, and network** measurements.
- Managed requirements, enhancements, deployment, troubleshooting, operational maintenance, and documentation throughout the product lifecycle.

Skills / Technologies: EPP, SLA analytics, Hadoop, Hive, Hive UDFs, OpenStack Swift, PostgreSQL, Grafana, historical metrics, production monitoring, root-cause analysis.

5.1.3 PSTools — Performance Testing Analytics and Release Reporting Portal

Description: Consolidated performance-testing reporting portal presenting load-test results, test SLA measurements, gateway/application performance, database SLA and system metrics, release reports, and EPP command latency distributions. PSTools is used to examine **performance deviations, bottlenecks, breaking points, and release degradations or improvements**.

Technical Contributions:

- Adapted the shared Sirius reporting framework for the **test environment**, with test SLA logs and metrics uploaded **hourly to a dedicated Hadoop cluster**.
- Automated Hive report generation, Swift publication, report monitoring, validation, and PostgreSQL loading for test measurements.
- Developed additional reports for load-test analysis, throughput behavior, performance limits, and software-release comparisons.
- Implemented consolidated **Grafana dashboards** combining load-testing SLA metrics, database metrics, application and gateway measurements, and historical test results.
- Developed release-comparison views and **EPP command response-time histograms** to expose regressions, improvements, latency shifts, and breaking points.
- Used the common PostgreSQL reporting data model with a **PSTools-specific schema**, preserving compatibility with the shared processing and ingestion components.
- Maintained and enhanced the reporting portal, deployment configurations, supporting environments, and documentation.

Skills / Technologies: Grafana, PostgreSQL, Hadoop, Hive, OpenStack Swift, EPP, performance testing, SLA metrics, database performance analysis, release regression analysis, visualization.

5.1.4 PSUTF — P&S Unified Testing Framework

Description: Designed and implemented a centralized, extensible load-generation framework for performance and scalability testing of Verisign registry products, including **Core, CTLD, and Name Suggestion**. PSUTF supports configurable, repeatable test scenarios that simulate representative production EPP workloads across supported TLDs, including **.com, .net, .cc, and .tv**.

Technical Contributions:

- Analyzed test requirements and designed and implemented the modular load-generation framework, including reusable commands and configurable data types.
- Developed extensible **command and data-bean components** using **Spring** and **Spring Expression Language (SpEL)**, allowing new commands and data types to be configured without modifying core framework logic.
- Implemented a centralized **JSON/REST configuration manager** and reusable scenario configuration.
- Developed **PSWeb**, a **GWT-based** interface displaying active test-run statistics through REST/JSON services.
- Implemented service-management scripts for the framework's **manager, agent, and console** components; developed **Ant** builds and **RPM** packaging specifications.
- Evaluated, installed, and configured **Rundeck** to automate installation, configuration updates, service-status checks, and test-environment maintenance.
- Maintained and enhanced workload simulation and testing infrastructure to reflect evolving production patterns and release-testing requirements.

Skills / Technologies: Java, Spring, SpEL, GWT, REST/JSON, Linux, Bash, Ant, RPM, Rundeck, EPP, workload modeling, distributed load generation, performance and scalability testing.

5.1.5 Infrastructure and Database Modernization

Description: Maintained and evolved the deployment, data-storage, and operational infrastructure for registry SLA reporting and performance-testing applications through successive infrastructure generations.

Virtual machines — Deployed, configured, maintained, and troubleshot application and reporting environments.

OpenStack — Developed **Terraform** provisioning and **Ansible** configuration/deployment automation; integrated reporting workflows with **Swift** object storage.

Verisign Kubernetes Services (VKS) — Developed Kubernetes manifests and **Helm charts** for deployment to Verisign's private Kubernetes clusters.

Database migration — Migrated an approximately **800 GB** reporting database from **MySQL to PostgreSQL**.

Cloud-native PostgreSQL — Transitioned PostgreSQL deployment to **CloudNativePG on Kubernetes**, preserving application and reporting integrations.

Distributed compute modernization — Initially established and managed a dedicated Hadoop reporting cluster; subsequently migrated reporting computation to the organization-wide **Verisign Shared Compute Cluster (VSCC)**, eliminating the need for a separately managed reporting cluster.

Visualization — Maintained **Grafana** reporting integrations and dashboards across infrastructure changes.

Technical Contributions:

- Planned and implemented deployment and configuration changes required to keep the reporting products operational across infrastructure migrations.
- Automated environment provisioning, configuration management, and application deployment using Terraform, Ansible, Kubernetes manifests, and Helm charts.
- Supported database migration and integration changes for large historical datasets and reporting pipelines.
- Established and administered the initial dedicated Hadoop reporting environment, then transitioned Hive reporting and related compute workloads to **VSCC** as shared organizational infrastructure became available.
- Maintained system operations, incident investigation, troubleshooting, enhancements, and technical documentation across the application lifecycle.

Skills / Technologies: Linux, virtual machines, OpenStack, Terraform, Ansible, Kubernetes, Helm, CloudNativePG, MySQL, PostgreSQL, Grafana, infrastructure automation, database migration, deployment engineering.

5.1.6 Domain History Migration — Oracle to Cassandra

Description: Migrated historical domain-transaction records from Oracle to Cassandra to support multiple reporting views over registry domain-history data.

Technical Contributions:

- Automated extraction and loading of historical transaction data using **Apache Sqoop**.
- Designed and implemented multiple **Apache Hive schemas**, using **bucketing** to organize data for reporting and analysis.

Skills / Technologies: Oracle, Cassandra, Hadoop, Hive, Sqoop, SQL, database migration, distributed data processing.

Consolidated Skills and Technologies

- **Registry and performance engineering:** EPP, registrar traffic analysis, SLA monitoring, cross-layer root-cause analysis, production workload modeling, load generation, performance/scalability testing, throughput and latency analysis, EPP histograms, release regression analysis.
- **Data engineering and reporting:** Hadoop, HDFS, Hive, Hive UDFs, Java MapReduce, Flume, Sqoop, OpenStack Swift, ETL, pluggable report frameworks, automated ingestion, Grafana, Pentaho BI.
- **Databases and infrastructure:** PostgreSQL, MySQL, CloudNativePG, Cassandra, Oracle, relational schema design, ~800 GB database migration, Linux, virtual machines, OpenStack, VKS/Kubernetes, Terraform, Ansible, Helm.
- **Application development and operations:** Java, Python, SQL, Spring, SpEL, GWT, REST/JSON, Bash, Rundeck, Ant, RPM, JProfiler, Grinder, requirements engineering, architecture, implementation, deployment, enhancements, operational support, and documentation.

5.2 Smith Micro Software, Inc.

Projects

5.2.1 SODA — Secure On-Device API

Description: SODA provided a common interface for secure device communications across mobile broadband devices, manufacturers, chipset suppliers, and operating systems. Contributed to **SODA-WEB**, the remote device-management interface, and **SODA-CA**, the certificate authority component responsible for establishing device and service identities.

Technical Contributions:

- Developed the **SODA-WEB dashboard** using **Google Web Toolkit (GWT)** and the **mvp4g** Model–View–Presenter framework to support remote management and configuration of SODA-enabled devices through REST services.
- Implemented **TLS mutual authentication** between SODA-WEB and the SODA server using OpenSSL APIs.
- Evaluated **Entrust Security Manager** and the Entrust Java Security Toolkit, and assessed open-source certificate authorities including **OpenCA** and **EJBCA** for integration with the SODA platform.
- Designed and implemented the **SODA-CA access layer** using EJBCA web services; extended EJBCA to support Smith Micro-specific X.509 certificate extensions.
- Designed and implemented the **certificate issuance workflow**, including key generation, **PKCS#8** private-key storage, **PKCS#10** certificate signing requests with custom extensions, issuance of **X.509v3 certificates**, and creation of **PKCS#12** credential bundles.
- Implemented a secure database API for accessing end-device **PKCS#12** information stored in **Cassandra**.

5.2.2 Mercury — Scalable Server Platform

Description: Mercury was a shared server platform designed to consolidate Smith Micro's server-side products into a scalable, highly available architecture supporting multiple applications.

Technical Contributions:

- Contributed to a **layered application architecture** designed to accommodate multiple server applications with different functional requirements.
- Implemented database-access layers for **memcachedb** and **Cassandra**.
- Developed administrative user-interface components using **GWT WindowBuilder**.
- Implemented **role-based access control (RBAC)** and strengthened host security using **iptables** and **SELinux** roles.
- Built an isolated performance-test environment and executed load tests with **JMeter** and **Grinder**; used **JProfiler** to identify application bottlenecks and execution hotspots.
- Tuned application-server, Linux operating-system, and JVM settings based on performance observations.

5.2.3 Cygnus — Secure Cloud File Delivery

Description: Cygnus was a hosted file-delivery service for securely distributing large documents. The service supported file transfers of up to **2 GB**, recipient-specific access without requiring recipients to maintain accounts, and **AES-256** encryption.

Technical Contributions:

- Implemented **Java Native Interface (JNI)** bindings to proprietary Smith Micro compression and encryption libraries.
- Designed the application database schema and implemented data access using **Cassandra** and the **Hector** client library.
- Contributed to the **REST-based file-upload interface**.
- Evaluated load-balancing and proxy technologies, including **Linux Virtual Server (LVS)**, **Squid**, **Varnish**, and **Zeus**.

- Installed and configured an **LVS** test environment with separate VLANs, network-interface bonding, and jumbo frames; performed load testing reaching **up to 4 Gbps across eight nodes**.

5.2.4 QuickLink Mobile VPN Gateway

Description: QuickLink Mobile VPN Gateway was a **MobileIP-based VPN solution** that maintained secure connectivity through dynamically established IPsec tunnels across changing networks and client devices. Work included gateway services, Linux and mobile clients, network appliances, authentication, availability, and operational tooling.

Technical Contributions:

- Designed and developed a **MySQL-based accounting and reporting subsystem** that recorded mobile-user sessions and traffic statistics and supported administrative reporting by user, department, access type, session duration, and bandwidth consumption.
- Implemented a **Linux kernel traffic-logging module** using **iptables/Netfilter** and network-flow information, and developed a **PHP-based administration interface**.
- Implemented **PostgreSQL-based real-time replication** of mobility-manager information to improve home-agent load balancing and mobile-node failover.
- Ported **MobileIP** code to **Windows Mobile** and **Windows XP** for the **3GPP2 FMC client**.
- Built customized Linux images for **QuickLink Mobility appliances (NXG500 and NXG5000)** and developed PHP-based appliance-configuration interfaces.
- Designed and implemented the **QuickLink VPN client for Linux**.
- Implemented a VPN-client simulator supporting **up to 5,000 simulated clients** for gateway stress testing.
- Enhanced MobileIP agent functionality, including a **multithreaded SSL server**, mobile-node configuration generation, address management, **IKEv2 integration**, and **IP-over-UDP kernel-driver** work.
- Contributed to load balancing, failover, and security improvements, including password protection, **TLS-secured intercomponent communication**, and authentication integration with **Microsoft Active Directory, OpenLDAP, RSA SecurID, and RADIUS**.
- Ported gateway software to the **Nokia IPSO** platform and resolved product defects and customer-reported issues.

5.3 Verizon Data Services Technology Centre

Projects

5.3.1 RequestNet — High-Capacity Service Pre-Provisioning

Description: RequestNet was an enterprise pre-provisioning system that provided a centralized web-based entry point for field sales and customer service personnel to submit and track requests for high-capacity telecommunications services. The system supported facility verification, request processing, and communication of service installation availability to the originating teams.

Technical Contributions:

- Developed and maintained **reporting modules** supporting the processing and visibility of service requests.
- Implemented enhancements to **critical-date calculation** functionality used in service-request workflows.
- Maintained and enhanced **QueryNet** modules within the RequestNet application.

5.4 ActivCard Software Private Limited

Projects

5.4.1 Smart Card Application Suite

Description: A cross-platform smart-card security suite for Linux, Solaris, and Mac OS X, supporting cryptographic operations, certificate management, user authentication, and smart-card issuance and maintenance.

Technical Contributions:

- Implemented smart-card middleware APIs (**BSI, XSI, and SCard**) to exchange **ISO 7816 Application Protocol Data Units (APDUs)** with smart cards.
- Implemented **PKCS#11 cryptographic services** for email encryption, digital signatures, secure browsing, and X.509 certificate download and import.
- Integrated smart-card authentication with the **Linux Pluggable Authentication Modules (PAM)** framework, supporting local authentication using passwords or PKI credentials and domain authentication using **Kerberos and Microsoft Active Directory**.
- Developed a database access API to retrieve smart-card lifecycle information from an **Oracle** backend for card issuance and updates through issuance and user portals.
- Developed testing utilities for **PKCS#11** and **Microsoft Cryptographic Service Provider (MS-CSP)** interfaces.
- Investigated **EMV authentication mechanisms** for potential support in the application suite.

5.5 Ericsson India Private Limited

Projects

5.5.1 Tigris — Multiservice Access Platform

Description: Ericsson Tigris was a family of high-capacity multiservice access platforms available in three-, seven-, and eleven-slot chassis configurations. It combined dial-in access, dedicated access, and wide-area routing with support for VPNs, IPsec, IKE, L2TP, RADIUS, VoIP, and Quality of Service (QoS) capabilities.

Technical Contributions:

- Integrated and tested an **IKE (Internet Key Exchange) stack from SSH** within Ericsson Datacom products.
- Contributed to **IPsec hardware acceleration using SafeNet ADSP-2141**, including separation of IPsec processing from IP processing, offloading packet handling to cryptographic hardware, and integration with the device-driver interface.
- Worked on **Security Association (SA) database management** within the cryptographic driver.
- Contributed to implementing **RFC 3193**, protecting L2TP tunnels with IPsec/IKE, including support for changing L2TP Network Server (LNS) IP addresses and port numbers through dynamic IPsec/IKE policies.
- Investigated **Simple Certificate Enrollment Protocol (SCEP)** and **Online Certificate Status Protocol (OCSP)** support for the Tigris platform.
- Implemented enhancements and resolved defects in the router software.

5.6 Bharat Electronics Limited (BEL) — Central Research Laboratory

Projects

5.6.1 FRED — Frame Relay Encryptor and Decryptor

Description: FRED secured Frame Relay traffic between a **Frame Relay Access Device (FRAD)** and a **Frame Relay Switch (FRS)** through encryption and decryption.

Technical Contributions:

- Contributed to implementing a **Linux cryptographic driver for the MOXA C101 PCI card** used in the Frame Relay encryption system.
- Worked with the card's high-speed synchronous/asynchronous interface, incorporating an **HD64570 Serial Communication Adapter (SCA)** and **256 KB of dual-ported RAM**.

5.6.2 GPS-Based Vehicle Tracking and Monitoring System

Description: A vehicle tracking and monitoring application that displayed a vehicle's current location using latitude and longitude data received from an onboard GPS receiver.

Technical Contributions:

- Implemented the **GPS module interface** to validate incoming GPS data and extract latitude and longitude coordinates from **81-byte GPS frames**.

6. Education and Certifications

6.1 Education

Master of Technology (M.Tech.) — Computer Science & Engineering
Regional Engineering College, Warangal

Master's Thesis: Public Key Encryption Systems at **Advanced Numerical Research and Analysis Group (ANURAG), Hyderabad**

6.2 Certifications

Cloudera Certified Developer for Apache Hadoop (CCD-410) — Exam passed September 2013; certification expired.